

Ödev & Notlandırma Sistemi

Veri Modeli

EduFlow için ödev, öğrenci cevabı, not ve geri bildirim yönetimini kapsayan MongoDB koleksiyon tasarımı.

6 koleksiyon

18 indeks

3 gömülü döküman

MongoDB 7.x

1 Koleksiyonlar ve Alanlar

users — kimlik, rol, oturum			
ALAN	TİP	KISIT	AÇIKLAMA
_id	ObjectId	PK	MongoDB otomatik üretilen birincil anahtar
name	String	Required	Tam ad
email	String	Required Unique	Giriş için kullanılan e-posta, unique indeks
passwordHash	String	Required	bcrypt hash, asla plaintext saklanmaz
role	String	Required	Enum: "student" "teacher" "admin"
avatarUrl	String	—	Profil fotoğrafı CDN URL'i
isActive	Boolean	default: true	Hesap aktiflik durumu (ban desteği)
createdAt	Date	Index	Kayıt tarihi
İNDEKSLER			
{ email: 1 } unique { role: 1 } { isActive: 1 }			

courses

— ders kataloğu

Alan	Tip	Kisit	Açıklama
<code>_id</code>	ObjectId	PK	Birincil anahtar
<code>teacherId</code>	ObjectId	FK → <code>users</code>	Dersi oluşturan öğretmenin referansı
<code>title</code>	String	Required	Ders başlığı (ör. "Fizik 101")
<code>code</code>	String	Unique	Kısa ders kodu (ör. "FIZ101")
<code>description</code>	String	—	Ders içeriği açıklaması
<code>enrolledStudents</code>	ObjectId[]	Index	Kayıtlı öğrenci ID listesi (hızlı yetki kontrolü için gömülü)
<code>startDate / endDate</code>	Date	—	Dönem başlangıç ve bitiş tarihleri
<code>isPublished</code>	Boolean	default: false	Öğrencilere görünürlük
<code>gradingScale</code>	Object	—	Gömülü not dönüşüm tablosu: { AA: 90, BA: 80, ... }
İndeksler			
{ code: 1 } unique			
{ teacherId: 1 }			
{ enrolledStudents: 1 }			

● assignments

— ödev tanımları



ALAN	TİP	KISIT	AÇIKLAMA
<code>_id</code>	ObjectId	PK	Birincil anahtar
<code>courseId</code>	ObjectId	FK → courses Index	Ait olduğu ders
<code>createdBy</code>	ObjectId	FK → users	Ödevi oluşturan öğretmen
<code>title</code>	String	Required	Ödev başlığı
<code>description</code>	String	—	Markdown destekli açıklama
<code>type</code>	String	Required	Enum: "homework" "quiz" "project" "exam"
<code>maxScore</code>	Number	Required	Tam puan değeri (ör. 100)
<code>weight</code>	Number	default: 1	Dönem notu hesabındaki ağırlık katsayısı
<code>dueDate</code>	Date	Index	Son teslim tarihi
<code>allowLate</code>	Boolean	default: false	Geç teslimi izin verilip verilmediği
<code>lateDeductionPct</code>	Number	—	Geç teslimde kesilecek puan yüzdesi
<code>maxAttempts</code>	Number	default: 1	İzin verilen maksimum gönderim sayısı
<code>attachments</code>	Object[]	—	Gömülü dizi: { filename, url, size, mimeType }
<code>published</code>	Boolean	default: false	Öğrencilere yayınlanma durumu
<code>publishAt</code>	Date	—	Zamanlanmış yayın tarihi

İNDEKSLER

{ courseId: 1, dueDate: 1 }

{ courseId: 1, published: 1 }

{ type: 1 }

GÖMÜLÜ: ATTACHMENTS DİZİSİ ÖRNEĞİ

```
attachments: [  
  {  
    filename: "soru_kagidi.pdf",  
    url:      "https://cdn.example.com/files/xyz.pdf",  
    size:     204800,           // byte  
    mimeType: "application/pdf"  
  }  
]
```

● **submissions** — öğrenci gönderileri

ALAN	TİP	KISIT	AÇIKLAMA
<u>_id</u>	ObjectId	PK	Birincil anahtar
assignmentId	ObjectId	FK → assignments	Hangi ödevle ait olduğu
studentId	ObjectId	FK → users	Gönderen öğrenci
status	String	Index	Enum: "draft" "submitted" "graded" "returned"
attemptNumber	Number	default: 1	Kaçıncı gönderim denemesi
submittedAt	Date	Index	Gönderim zaman damgası
isLate	Boolean	—	Son tarihi geçip geçmediği
files	Object[]	—	Gömülü: { filename, url, size, mimeType }
textAnswer	String	—	Metin tabanlı cevaplar için zengin metin alanı
grade	Object	—	Gömülü: { score, maxScore, letterGrade, gradedAt, gradedBy }
feedback	Object	—	Gömülü: { general, rubricScores[], privateNote }

İNDEKSLER

{ assignmentId: 1, studentId: 1 } unique (per attempt)

{ studentId: 1, status: 1 }

{ assignmentId: 1, status: 1 }

{ submittedAt: -1 }

GÖMÜLÜ: GRADE VE FEEDBACK YAPISI

```
grade: {
  score: 87,
  maxScore: 100,
  letterGrade: "BA",
  gradedAt: ISODate("2025-11-20T14:30:00Z"),
  gradedBy: ObjectId("...") // öğretmen ID'si
},
feedback: {
  general: "Kavramsal yaklaşım doğru, ancak...",
  privateNote: "Öğrenciye ek kaynak önerilecek", // sadece öğretmene görünür
  rubricScores: [
    { criterionId: ObjectId("..."), score: 18, comment: "İyi" }
  ]
}
```

grades	— dönem not defteri			
Alan	Tip	Kisit	Açıklama	
<code>_id</code>	ObjectId	PK	Birincil anahtar	
<code>courseId</code>	ObjectId	FK → <code>courses</code>	İlgili ders	
<code>studentId</code>	ObjectId	FK → <code>users</code>	Öğrenci	
<code>submissionId</code>	ObjectId	FK → <code>submissions</code>	Kaynağı olan gönderi	
<code>gradedBy</code>	ObjectId	FK → <code>users</code>	Notlayan öğretmen (denetim izi)	
<code>score</code>	Number	Required	Alınan ham puan	
<code>maxScore</code>	Number	Required	Tam puan değeri	
<code>percentage</code>	Number	—	Önceden hesaplanmış yüzde (aggregation maliyetini düşürür)	
<code>letterGrade</code>	String	—	Harf notu: AA, BA, BB, CB, CC, DC, DD, FF	
<code>gradedAt</code>	Date	Index	Notlama zaman damgası	
<code>isFinalized</code>	Boolean	default: false	Öğretmen notu kesinleştirdi mi	
<code>appealStatus</code>	String	—	Not itiraz durumu: "none" "pending" "resolved"	
İndeksler				
{ <code>courseId</code> : 1, <code>studentId</code> : 1 }				
{ <code>studentId</code> : 1, <code>gradedAt</code> : -1 }				
{ <code>courseId</code> : 1, <code>isFinalized</code> : 1 }				
{ <code>submissionId</code> : 1 } unique				

● rubrics — değerlendirme kriterleri

ALAN	TİP	KISIT	AÇIKLAMA
<code>_id</code>	ObjectId	PK	Birincil anahtar
<code>assignmentId</code>	ObjectId	FK → assignments Unique	Her ödevin en fazla bir rubric'i olabilir
<code>createdBy</code>	ObjectId	FK → users	Rubric'i hazırlayan öğretmen
<code>criteria</code>	Object[]	Required	Değerlendirme kriterleri dizisi (gömülü)
<code>totalPoints</code>	Number	Required	Tüm kriterlerin toplam puanı

GÖMÜLÜ: CRİTERİA DİZİSİ YAPISI

```
criteria: [  
  {  
    _id: ObjectId("..."),  
    title: "Kod kalitesi",  
    description: "Okunabilirlik, yorum satırları",  
    maxPoints: 30,  
    levels: [ // puanlama seviyeleri  
      { points: 30, label: "Mükemmel", description: "..." },  
      { points: 20, label: "İyi", description: "..." },  
      { points: 10, label: "Gelişmeli", description: "..." }  
    ]  
  }  
]
```

İNDEKSLER

```
{ assignmentId: 1 } unique
```

2

İlişkiler ve Kardinalite

users → courses

1:N

Bir öğretmen birden fazla ders açabilir.
`courses.teacherId` referans tutar.

courses → assignments

1:N

Bir ders birden fazla ödev içerir.
`assignments.courseId` ile bağlı.

assignments → submissions

1:N

Her ödev binlerce gönderi alabilir. Aynı koleksiyonda tutulur, gömülü değil.

submissions → grades

1:1

Her gönderinin en fazla bir kesinleşmiş notu vardır.
`grades.submissionId` unique indeksli.

assignments → rubrics

1:1

Bir ödev en fazla bir rubric'e sahip olabilir. Kriterlerin kendisi rubric içine gömülüdür.

courses → users (öğrenci)

N:M

courses.enrolledStudents dizisine gömülü ID listesi. Kayıt sayısı fazlaysa ayrı enrollments koleksiyonu önerilebilir.

3

Tasarım Kararları

Not bilgisi submissions'a gömülü, grades ayrı koleksiyonda

Öğrenci kendi gönderisini açtığında hızlı okuma için `submissions.grade` alanı anlık veriyi taşır. `grades` koleksiyonu ise öğretmenin tüm sınıf not defterini görüntülemesi, itiraz yönetimi ve dönem sonu hesaplamaları için ayrı tutulur. Bu çift yazma stratejisi okuma hızını artırır.

Rubric kriterleri gömülü, rubric'in kendisi ayrı koleksiyonda

Bir rubric'in kriterleri sıklıkla birlikte okunur, bu yüzden `criteria` dizisi gömülüdür. Ancak rubric'in kendisi `assignments`'a gömülmez; çünkü şablon olarak kopyalanabilir ve bağımsız sorgulanabilir olması gerekir.

submissions.feedback.privateNote öğrenciye gönderilmez

API katmanında `feedback.privateNote` alanı öğrenciye asla dönmez (projection ile filtrelenir). Bu sayede öğretmen özel notlar alabilir. Alternatif: ayrı `teacherNotes` koleksiyonu kullanılabilir.

grades.percentage önceden hesaplanmış

`score / maxScore * 100` her okumada hesaplamak yerine not yazılırken saklanır. Sınıf ortalaması, dağılım grafikleri gibi istatistikleri aggregation pipeline'da `$avg` ile anında hesaplamayı mümkün kılar.

Multi-attempt desteği için submissions benzersiz indeksi

`{ assignmentId, studentId, attemptNumber }` bileşik unique indeksi, bir öğrencinin aynı denemede iki kayıt oluşturmasını engeller. `maxAttempts` kontrolü API katmanında yapılır.



Bileşik indeksler

En sık çalışan sorgu desenleri (`courseId + dueDate`, `studentId + status`) bileşik indeksle tek adımda çözülür. Collection scan'ın önüne geçilir.



Seçici gömme

Sık birlikte okunan küçük veriler (`attachments`, `grade özeti`) gömülür; büyüyecek veriler (tüm gönderi listesi) referans olarak tutulur. 16 MB döküman sınırı aşılmaz.



Sharding hazırlığı

`submissions` ve `grades` koleksiyonları `courseId` shard key'i ile yatay bölünebilir. Her ders kendi veri segmentinde tutulur.



Covered queries

İndeks alanları sorgu + projeksiyon'u karşıladığında MongoDB dökümanı okumadan yanıt verir. Özellikle `status` sayım sorgularında etkili.



Aggregation pipeline

Sınıf not dağılımı, ödev tamamlanma oranı gibi raporlar için `$match` → `$group` → `$project` pipeline kullanılır. `allowDiskUse: true` büyük veri setleri için açık tutulur.



TTL indeksleri

Taslak (draft) gönderiler belirli süre sonra otomatik silinebilir. `submittedAt` üzerine TTL indeksi eklenerek eski taslaklar MongoDB tarafından temizlenir.